Fit Check

Build It, Then Measure It · Capstone Team 14 · NBAY 6170 · November 13, 2026

> **Fit Check tells an online shopper, in under a minute, whether their usual size of a clothing item is likely to fit, using what real reviewers wrote.** It is a working, API-agnostic web app: five AI agents read the product page, collect more reviews from other stores, check the brand, tag every review and make the call. Every warning carries the reviewers' exact words. It never tells anyone which size to buy.

20 of 20	**10 of 10**	**$0.022**	**$0.18**
golden cases scored 5 of 5 on the larger model tier (small tier: 3.4 average)	agreement between our scores and an independent model judge	cost per full check, measured tokens × list prices (~41 s median)	value per purchase in avoided fit returns (base case)

1. Working prototype

Live app: varunkulkarni0995-svg.github.io/fit-check (v0.16.1) · **Code and README:** github.com/varunkulkarni0995-svg/fit-check · The single-file prototype (index.html) is attached. Test pages with expected results are in the repo's *testdata* folder.

Agent	Job	Default model (backup)
Reader	Reads the uploaded PDF, screenshot or page: product, size chart, every review. Long uploads are read in parts.	Groq gpt-oss-20b (Gemini 3.5 Flash)
Review collector	Finds the same product on other stores and in the team's review memory; reads review data directly when stores publish it.	Serper + Firecrawl; Groq gpt-oss-120b
Web researcher	Brand and seller complaints from the last 6 months.	Serper + Firecrawl; Groq gpt-oss-20b
Analyst	Tags each review: how it runs, problems with exact quotes, body words, brand comparisons. **The app does the counting in code.**	Groq gpt-oss-120b (Gemini 3.5 Flash)
Decider	Applies the evidence rules and writes the result card.	Groq qwen3.8-27b (Gemini 3.5 Flash)

Any provider can be swapped per agent in Settings (Groq, Gemini, Anthropic, OpenAI, OpenRouter, Mistral and others). Every request, response and button tap is logged to a team Google Sheet with a KPIs tab. Guardrails: review text is treated as data (prompt-injection cases pass), quotes are verified word for word, and no size advice is given.

2. Simplified workflow

TODAY: THE SHOPPER DOES THE ANALYSIS

1. Open reviews Hundreds of ratings, a few shown at a time → **2. Scroll and filter** Hunt for people with a similar size and body → **3. Weigh mixed opinions** "Runs small" vs "fits great" → **4. Guess a size** Often order two sizes → **5. Return** Size and fit drive most apparel returns

WITH FIT CHECK: THREE STEPS, UNDER A MINUTE

1. Profile once Height, usual size, body type, brands that fit → **2. Upload the page** PDF or screenshot; reviews from other stores are added → **3. Read the call** Good fit / Check first / Poor fit, with quotes, counts and confidence

The agents do the slow parts (reading, finding comparable shoppers, counting, checking the brand). Where reviewers disagree, Fit Check shows both sides instead of averaging them away.

3. Knowledge source

Customer reviews, retrieved per product, are the knowledge source. They come from three places: the uploaded page, the same product's pages on other stores (searched and read live), and a shared team review memory that keeps every review extracted in earlier checks (Google Sheet, looked up by brand and product). A brand-and-seller web search for the last 6 months and the product's size chart add context.

Why we chose it: reviews are the only evidence of how an item actually fits real bodies; a size chart describes the garment, not the experience. Retrieval keeps the evidence specific to this product and current, and every claim stays traceable: each warning shows how many reviewers said it and their exact words, which the app checks against the source text. The memory makes the system better with use, which is also the start of our moat.

4. Eval pack (attached: Fit_Check_Eval_Pack.xlsx)

20 golden cases with expected outputs: D01–D10 test the final fit call (Decider); A01–A10 test review analysis (Analyst). They include traps: too few similar reviews, conflicting reviews, hidden instructions in reviews, spam, unclear fabric and exact threshold boundaries. A full 20-case Analyst set and a 6-upload Reader set are in their own tabs.

Measure	Result
Team score, 1–5, golden 20 (larger tier)	20 of 20 scored 5. Decider run live on Nov 13 with the app's own models; Analyst on Nemotron 3 Ultra.
Same 20 on the small tier (Gemini Flash-Lite)	3.4 average; 7 of 20 scored 5
Model as judge (Groq gpt-oss-120b, a different model) on the 10 Decider cases	Final run: 10 of 10 exact agreement. First run: 9 of 10 exact, including 5 failed runs both scored 1. The one disagreement was the judge correctly catching a bug in our own scorer (it read a quoted "size up" as advice).

Top three failure patterns, and what we changed

Pattern	Evidence	Fix
1. Small models break fixed rules and miscount	Flash-Lite: 7 of 10 right calls (a warning at 12.5% share; a mislabelled issue; the 3-warning cap broken). Picked the right similar reviews only 62% of the time.	Counting, matching and thresholds moved into code (v0.14). The model only tags.
2. Long uploads overflow the model	Real 40–60 review PDFs: answers cut off at the output limit; Groq free tier refused requests over 8,000 tokens a minute.	Read in parts split at review boundaries, then merged (v0.16): 60 of 60 reviews read in testing.
3. Too little usable evidence on real pages	A product page shows about 8 reviews; most omit size or height; some stores hide reviews in widgets.	Review-page upload tip, other-store reviews, review memory, and a clearly labelled all-shopper fallback.

5. Unit economics (attached: Fit_Check_Unit_Economics_Executive_Report.pdf)

The same 10 Decider tasks on two model tiers

Model	Right calls	Avg latency	Cost per 100 runs
Gemini 3.5 Flash (larger tier)	10 of 10	12.1 s	$2.08
Gemini 3.5 Flash-Lite (small tier)	7 of 10	5.0 s	$0.15
Groq qwen3.8-27b (app default, live run Nov 13)	10 of 10	2.5 s (3.8 s incl. 2 backup runs)	$0.28

Trade-off we accept: we pay for a larger tier on the judgement steps. Flash-Lite saves about 2 cents a decision, but a wrong call either hides a real fit problem or invents one; if just 1 in 233 wrong calls causes a $15 return, the

saving is gone. We also removed the trade-off at its root by moving the fixed rules into code, so a mid-size, fast model is enough.

Cost, speed and value

Metric	Value	Basis
Cost per full check	$0.022 (AI $0.007 + search and scrape $0.012, +15% for retries)	Median tokens from our logs × list prices; the team report estimated $0.026
Time per check	~41 s median (9 live checks)	Free tiers add rate-limit waits; paid tiers and caching cut this
Value per purchase	$0.18 base case	23.4% return rate × 50% due to fit × 10% prevented × $15 to process a return
Value ÷ cost	2.3× shopper-triggered; 120× run once per product and cached; loses money if run on every page view	Team report, section 6
Guardrail	Lost sales above 0.64% erase the value	We test conversion as well as returns, and never say "don't buy"

6. Commercial

Customer: apparel retailers and brands, who pay for returns; shoppers use it free. **Offer:** fit checks on the retailer's product pages, computed once per product from its reviews and refreshed as new reviews arrive. **Pilot price to test:** $2 per product per month for products with 10 or more reviews, about 23% of the avoided-return value such a product creates in the base case. A retailer with 5,000 such products would pay $10,000 a month for about $366 of compute (team report).

Question	Our answer
Why now	Per-check AI cost is a couple of cents, so the business is decided by architecture (cache per product) and accuracy, not token prices.
Moat	A growing cross-store review memory, "fits like a brand you wear" comparisons across brands, our eval set, and later the return outcomes tied to each call. A competitor can copy the prompts in a weekend, not the data.
Amazon angle	Large marketplaces can build fit features in-house, so we do not depend on scraping them (we never do). Paths: sell to brands and retailers off Amazon; or partner where official review-data access exists. With a review-data feed, uploads and scraping disappear and the cost per check falls to model cost, while per-size fit scores become possible.
Go to market	Start with direct-to-consumer apparel brands with many reviews per product; prove fewer fit returns in an A/B test with a 0.5% conversion guardrail; then expand to multi-brand retailers.
Key risks	Wrong or alarming calls (mitigated by evidence rules, quotes and evals); review data terms (official feeds only at scale); free-tier limits and preview models (paid tiers and backups for a pilot).

Sources: team unit-economics report (Nov 13, 2026: Coresight return rates, Optoro return costs, provider list prices); Fit Check logs (team Sheet); team eval kits for the Reader, Analyst and Decider (Nov 12–13, 2026). Built with Claude (orchestration and front end), API Bench for model tests, and Gemini for unit-economics research.